

A Method of One-Pass Seed Generation for LFSR-Based Deterministic/Pseudo-Random Testing of Static Faults

Takanori Moriyasu

Graduate School of Engineering, Oita University
700 Dannoharu, Oita 870-1192, Japan
Email: v13e3026@oita-u.ac.jp

Satoshi Ohtake

Faculty of Engineering, Oita University
700 Dannoharu, Oita 870-1192, Japan
Email: ohtake@oita-u.ac.jp

Abstract—This paper proposes a method of LFSR seed generation for deterministic and pseudo-random testing of static faults. The proposed method directly generate seeds by using ATPG and avoids unsuccessful encoding of conventional two-pass generation methods. The effectiveness of the proposed method is evaluated through experiments for several LFSR-based pseudo-random pattern generators. The quality of seeds generated by the proposed method is evaluated by stuck-at fault coverage when test patterns are expanded from seeds.

Keywords—LFSR seed generation; pseudo-random test; deterministic test; scan-based BIST; phase shifter; constrained ATPG;

I. INTRODUCTION

Recently, the size of logic circuits implemented on large-scale integration (LSI) circuits has come to become huge according to advances in LSI technology. Since the test cost increases with increase of the size of logic circuits, reducing the cost has become more important. Built-in self-test (BIST) is widely used to reduce cost of manufacturing test and to enable in-field test [1]. BIST is a design technique in which a test pattern generator (TPG) and a response analyzer (RA) are inserted into a circuit under test (CUT). On typical scan-based BIST, a CUT is fully scannable and a TPG which drives primary inputs and scan inputs of the CUT and an RA which is driven by primary outputs and scan outputs of the CUT are formed with a linear feedback shift register (LFSR) and a multiple input signature register (MISR), respectively.

An LFSR can generate a pattern sequence pseudo-randomly from an initial value called a seed and can generate all the patterns except the all zero pattern if its feedback polynomial is primitive. However, pseudo-exhaustive test is usually not possible and such pseudo-random pattern test cannot always achieve high fault coverage [1]. Existence of random pattern resistant faults (RPRFs) is the major cause of the fault coverage saturation. An RPRF is a fault which is difficult to be detected without using some specific pattern. To detect such a fault, LFSR reseeding is known to be effective. In a reseeding process, a new seed is set to the LFSR so that some RPRFs are detected by the patterns expanded from the seed.

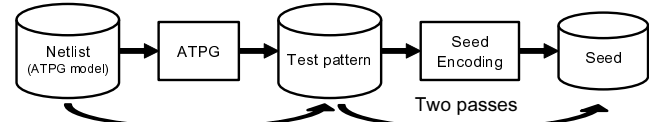


Figure 1. Flow of conventional seed generation (two passes).

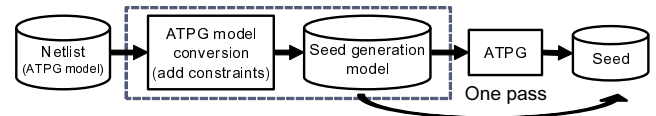


Figure 2. Flow of our proposed seed generation (one pass).

There exist several conventional seed generation methods for reseeding: some seeds are randomly generated [2], and test patterns detecting RPRFs are first generated using automatic test pattern generation (ATPG) tool (1st pass) and each generated test pattern is then encoded into a seed (2nd pass) [3][4]. However, the former type does not guarantee to detect RPRFs. The processing flow of the latter type is shown in Figure 1. In this paper, this type of a method is referred to as a *two passes* seed generation method. The two passes method does not always succeed in the encoding and may not achieve satisfactory fault coverage. In addition, the number of test patterns is large because test generation includes don't cares to improve encodability. However there are a number of works based on the two passes method to improve test quality, introduction of these works are omitted here due to the page limitation.

This paper proposes a new seed generation method for removing the drawbacks. On scan-based BIST, a test pattern, which will be loaded into scan paths and applied to the primary inputs, generated from an LFSR can be represented by the functions of the seed variables of the LFSR. The conventional seed generation method encodes a test pattern into a seed based on these functions. The proposed method generates LFSR seeds directly using an ATPG tool with ATPG constraints which represents those functions as shown in Figure 2. In this paper, this type of a method is referred

to as a *one pass* seed generation method. In the proposed method, a seed generation model is first created by connecting an XOR network, which is a combinational circuit representing those functions, and the combinational part of the CUT. Seeds are then generated as test patterns for the seed generation model using an ATPG tool.

There exists a work using the one pass concept [5]. In that work, the structure of a broadcasting network placed on scan inputs (virtual scan mechanism) is considered during test generation so that a test pattern loaded into the scan paths by way of the broadcasting network guarantees to detect dedicated faults.

The authors have proposed a basic concept of this seed generation method in [6]. The method was designed by assuming single test pattern expansion from each seed and does not support phase shifters, which are usually used with LFSRs in BIST. In this paper, the method is augmented to support phase shifters and evaluated for not only single pattern expansion from each seed but also multiple test pattern expansion from each seed. The effectiveness of the augmented method is evaluated for an LFSR with and without a phase shifter.

This paper is organized as follows. Section II explains the scan-based BIST model in this paper. Section III introduces a conventional seed generation method and discusses its problems. In Section IV, our seed generation method is proposed, and Section V shows its effectiveness through experiments. Finally, Section VI concludes the paper.

II. TARGET BIST MODEL

A. LFSR based BIST

A BIST model targeted in this paper is shown in Figure 3. It consists of an LFSR, a phase shifter, a circuit under test (CUT) and a MISR, where a CUT is fully scannable.

In BIST mode operation, a seed is loaded into the LFSR, the LFSR is run until scan FFs are fulfilled (test pattern expansion) with scan mode, single system clock is applied for testing static faults and then the response loaded into the scan FFs is scanned out to the MISR for compressing. In this paper, we consider single test pattern expansion per seed and multiple test pattern expansion per seed. The former expand only one test pattern for a seed, which will be utilized for test volume reduction of deterministic test, and the latter expand multiple test patterns for a seed, which will be used for standard reseeding based BIST.

B. Random pattern resistant fault

Consider an AND gate with n -inputs. For the AND gate, the all-1 pattern can only test the stuck-at-0 fault on the output. When applying unbiased random inputs, the probability of justifying 1 on the output is $1/2^n$. Application of pseudo-random patterns generated by an LFSR is difficult to detect such a fault. Such a fault is called a random pattern resistant fault (RPRF).

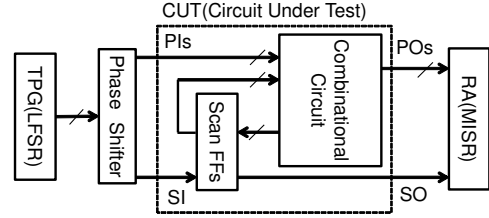


Figure 3. A model of scan-based BIST.

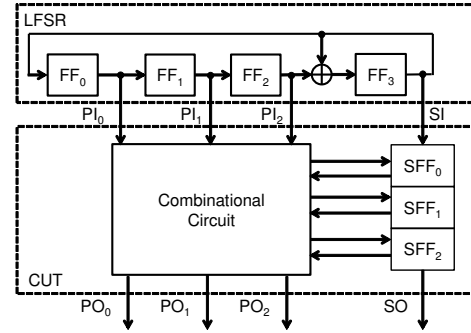


Figure 4. An example of CUT and LFSR.

III. CONVENTIONAL METHOD

Given a set of deterministic test cubes, a conventional way is to compute a set of seeds that when expanded by the LFSR will produce the deterministic test cubes. For each test cube, a seed can be computed by solving a system of linear equations based on the feedback polynomial of the LFSR[3].

For example, consider an example circuit composed of an LFSR of 4 stages and a CUT with primary inputs, PI_0 , PI_1 and PI_2 , and scan flip-flops (FFs), SFF_0 , SFF_1 and SFF_2 , shown in Figure 4. A test pattern is generated from the LFSR by applying three cycles under scan shift mode where the three cycles corresponds to the length of the scan path. Suppose that the seed of the LFSR is $(FF_0, FF_1, FF_2, FF_3) = (S_0, S_1, S_2, S_3)$. Each coordinate of the expanded pattern can be represented by the function of the seed. In this case, $(PI_0, PI_1, PI_2, SFF_0, SFF_1, SFF_2)$ is $(S_2 \oplus S_3, S_3, S_0, S_1 \oplus S_2 \oplus S_3, S_2 \oplus S_3, S_3)$ as shown in Figure 5.

In the conventional method, a test cube $(PI_0, PI_1, PI_2, SFF_0, SFF_1, SFF_2) = (T_0, T_1, T_2, T_3, T_4, T_5)$ is generated and encoded into a seed by solving the following system of linear equations: $(T_0, T_1, T_2, T_3, T_4, T_5) = (S_2 \oplus S_3, S_3, S_0, S_1 \oplus S_2 \oplus S_3, S_2 \oplus S_3, S_3)$. These linear equations can be solved by using Gauss-Jordan elimination or SAT solver.

Although this method is widely used so far, there is a drawback that the encoding does not always succeed. It means that it may induce fault coverage loss. An ATPG tool may not generate a test cube, which can be encoded into a seeds, for a detectable fault of a CUT under the BIST

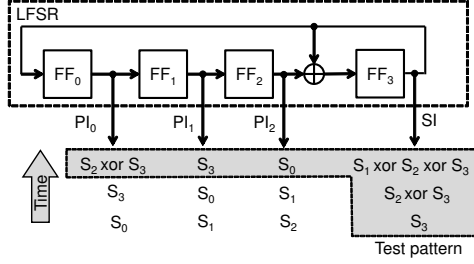


Figure 5. A test pattern expanded by LFSR from a seed.

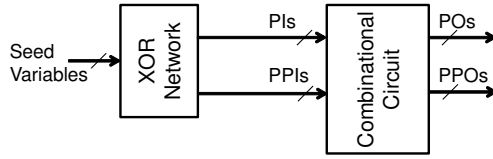


Figure 6. The proposed seed generation model.

model because the ATPG tool does not know its following process.

For example, a test cube $(1, 1, X, 0, 0, 0)$ is generated by an ATPG tool. This pattern can not be encoded into a seed because the system of linear equations $(1, 1, X, 0, 0, 0) = (S_2 \oplus S_3, S_3, S_0, S_1 \oplus S_2 \oplus S_3, S_2 \oplus S_3, S_3)$ is unsolvable.

Furthermore, we need to emphasize that we must run fault simulation for test patterns expanded from the seeds generated by this conventional method if the quality of the seeds needs to be evaluated.

IV. PROPOSED METHOD

To remove the drawback of the conventional method, we propose a method to generate LFSR seeds directly using an ATPG tool. The main idea of the method is to use an *XOR network*, which is obtained by time frame expansion of the LFSR, as a constraint for ATPG. The number of time frames for the expansion is equal to the length of the longest scan path in the CUT. The XOR network represents the relation between a seed and a test pattern for the primary inputs and scan FFs of the CUT. To inform the relation to the ATPG tool, we create a *seed generation model* which consists of the combinational part of the CUT and the XOR network as shown in Figure 6.

Consider the example circuit of Figure 4 again. The LFSR can be expanded with three time frames from a seed as Figure 5. For testing the CUT, only the shaded part is meaningful. The part can be represented by a combinational circuit (an XOR network) as shown in Figure 7. A seed generation model is created by connecting the XOR network and the combinational part of the CUT as shown in Figure 8. By targeting a fault in the combinational part, an ATPG

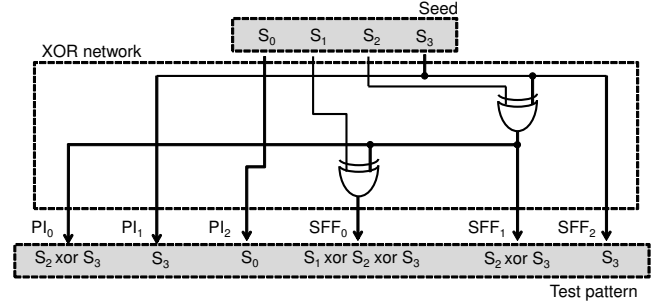


Figure 7. An XOR network representing 3-time expansion of LFSR.

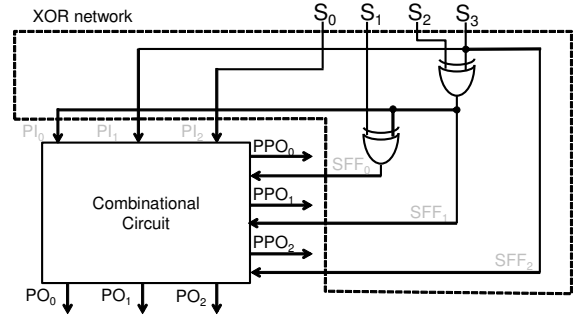


Figure 8. A seed generation model.

tool can generate a seed that when expanded by the LFSR will detect the fault or can prove that the fault is untestable.

The proposed method has advantages that an ATPG tool can generate all the LFSR seeds for detectable faults by the BIST model and calculate fault coverage and fault efficiency. So it is not necessary to perform another fault simulation for test patterns expanded from the obtained seeds.

In this paper, we target the scan-based BIST with a phase shifter as shown in Figure 3. The phase shifter can also be considered in our seed generation model. Basically, a pattern generation circuit composed of an LFSR and a phase shifter is a sequential circuit with FFs and XORs. In the seed generation model, we can also create an XOR network by n time frame expansion of the pattern generation circuit where n is the length of the longest scan path. The phase shifter part, which is called a *phase shifter network*, of the XOR network can be separately designed by composing of n copies of the phase shifter and placed in parallel as shown in Figure 9.

Notice that the seed generation model is only used during seed generation or no modification is made for the physical design of the circuit.

V. EXPERIMENTAL RESULTS

We evaluate the effectiveness of our proposed method by experiments. In the experiments, we used ITC'99 benchmark

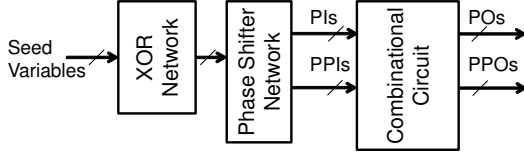


Figure 9. The proposed seed generation model with phase shifter network.

Table I
CHARACTERISTICS OF ITC'99 BENCHMARK CIRCUITS.

Circuit	# PIs	# POs	# Gates	# FFs	# SPs	SL
b14	32	54	8,567	245	3	82
b15	36	70	7,922	449	5	90
b17	37	97	27,852	1,415	24	59
b18	37	23	94,249	3,320	3	1,107
b19_6	24	30	190,213	6,642	6	1,107
b19_13					13	511
b19_22					22	302
b20	32	22	17,158	490	5	98
b21	32	22	17,482	490	17	30
b22	32	22	25,460	735	5	147

circuits [7]. Multiple scan paths were inserted in each circuit. The characteristics of each circuit is shown in Table I. Columns “# PIs”, “# POs”, “# Gates”, “# FFs”, “# SPs” and “SL” show the numbers of primary inputs, primary outputs, gates, FFs and scan paths and the length of the longest scan path, respectively. For b19, which is the largest one, we prepared three circuits with the different numbers of scan paths.

We extracted RPRFs for each circuit. We used two types of BIST structures with an LFSR of 100 stages and the LFSR of 100 stages with a phase shifter for every circuit. The feedback polynomials for these LFSRs were selected from [8]. We applied 10,000 pseudo-random patterns generated from some seed for stuck-at faults using the fault simulator of TetraMAX (Synopsys). We assumed that the remaining faults are RPRFs which will be targeted for seed generation for each BIST structure. Table II shows the number of RPRFs for each circuit of each BIST structure. Columns “Total # faults” and “100 stages” and “100 stages w/ PS” under “# RPRFs” show the numbers of faults and the numbers of RPRFs for the LFSR of 100 stages and the LFSR of 100 stages with the phase shifter, respectively. The numbers of RPRFs were not very differ from each other for each circuit.

The size of the XOR network is shown in Table III. Columns “Size (# XOR gates)” shows the number of XOR gates composing the XOR network for each BIST structure. Column “100 stages w/ PS” include the size of the phase shifter network. A number of XOR gates are removed based on the characteristic of the XOR function during time frame expansion. Therefore, the size of the XOR network becomes not so large.

We implemented the conventional method using a SAT

Table II
THE NUMBER OF RPRFs OF ITC'99 BENCHMARK CIRCUITS.

Circuit	Total # faults	# RPRFs	
		100 stages	100 stages w/ PS
b14	60,080	7,469	7,418
b15	55,586	11,389	13,586
b17	197,992	69,638	61,652
b18	687,576	192,571	201,326
b19_6	1,385,822	413,297	415,827
b19_13		410,826	406,513
b19_22		415,265	400,606
b20	120,882	10,751	12,987
b21	123,220	15,377	15,299
b22	179,614	16,779	16,705

Table III
THE SIZE OF THE XOR NETWORK.

Circuit	Size (# XOR gates)	
	100 stages	100 stages w/ PS
b14	53	304
b15	67	556
b17	56	1,457
b18	1,887	11,477
b19_6	1,843	6,909
b19_13	602	12,929
b19_22	313	8,844
b20	71	659
b21	15	490
b22	120	988

solver and employed MiniSat [9]. We generated seeds for RPRFs for the BIST structures by the conventional method and the proposed method. Tables IV and V show the results of the conventional method and the proposed method for the LFSR of 100 stages, respectively. In Table IV, columns “ATPG”, “Encode” and “CPU(sec.)” show the result of the ATPG for RPRFs, the results of the encoding from the test patterns and CPU time required for ATPG and encoding, respectively. Columns “%FC”, “%FE”, “# patterns” and “CPU(sec.)” under “ATPG” show fault coverage, fault efficiency, the number of test patterns with *don't care* (cubes) obtained by ATPG and CPU time required for ATPG, respectively. Columns “%FC” and “# Seeds” under “Encode” show fault coverage and the number of seeds. The fault coverage was obtained by fault simulation using the test patterns expanded based on single test pattern expansion per seed. In Table V, columns “%FC”, “%FE”, “CPU(sec.)” and “# Seeds” show fault coverage, fault efficiency, CPU time required for ATPG, and the number of seeds generated as test patterns, respectively. In the experiments, TetraMax was also used as an ATPG tool. The abort limit was 10 seconds for each faults in this experiment. The CPU used in the experiments was Xeon X5675 3.06GHz (Intel) with 40GB memory. From the results, we showed that our proposed method can achieve higher fault coverage for every circuit with comparable CPU time compared with the conventional method.

Table IV
RESULTS OF THE CONVENTIONAL METHOD FOR 100 STAGES LFSR.

Circuit	ATPG				Encode		CPU (sec.)
	%FC	%FE	# Patterns	CPU(sec.)	%FC	# Seeds	
b14	96.09	99.87	945	0.40	36.16	206	0.80
b15	85.09	91.76	1,295	2.13	59.69	733	2.70
b17	92.75	96.09	8,330	9.61	47.40	2,336	13.01
b18	97.90	99.30	23,583	21.64	95.22	22,541	34.51
b19_6	96.45	97.99	50,376	78.31	91.36	46,772	100.43
b19_13	96.42	97.97	49,991	84.46	92.26	46,872	103.53
b19_22	96.48	98.01	50,438	123.67	86.29	43,784	142.65
b20	94.74	99.86	1,375	0.86	39.35	349	1.31
b21	94.75	98.86	1,711	0.96	3.15	56	1.73
b22	95.78	99.87	1,949	1.13	58.06	841	1.94

Table V
RESULTS OF THE PROPOSED METHOD FOR 100 STAGES LFSR.

Circuit	%FC	%FE	CPU(sec.)	# Seeds
b14	96.06	99.87	0.45	660
b15	74.75	90.82	1.83	447
b17	81.63	94.71	7.53	2,121
b18	97.07	98.43	20.25	8,896
b19_6	94.73	96.43	86.80	17,285
b19_13	95.98	97.74	63.07	17,941
b19_22	93.32	97.74	65.32	17,594
b20	90.30	99.37	0.87	1,012
b21	76.33	97.72	3.77	941
b22	95.21	99.13	1.30	1,455

For the LFSR of 100 stages with a phase shifter, we also applied the conventional method and the proposed method. In this experiment, phase shifters were designed by following the method proposed in [10]. For each circuit, the number equals to the length of the longest scan paths was set to the size of the channel separation in the algorithm. The RPRFs under this structure were also obtained as shown in Table II. Tables VI and VII show the results of the conventional method and the proposed method for the LFSR with the phase shifter, respectively. For b18, b19_6 and b19_13 (circuit name with *), the conventional method with the SAT solver was not able to encode the test patterns into seeds in realistic time. For these circuits, we tried to employ Gauss-Jordan elimination [11] for encoding instead of the SAT solver. For these circuits, CPU time includes system time for the process. For a problem with small instances, the SAT solver was much faster than our implementation of the Gauss-Jordan elimination with Perl script language, but, for these circuits, the Gauss-Jordan elimination could solve the problem. The conventional method with the Gauss-Jordan elimination achieved higher fault coverage than the proposed method. However the proposed method was able to reduce the number of seeds and shorter generation time compared with the conventional method. For the other circuits except for b19_22, the proposed method achieved higher fault coverage compared with the conventional method. For b19_22, although the fault coverage is slightly low compared with the conventional method, the proposed method reduced the

Table VI
RESULTS OF CONVENTIONAL METHOD FOR 100 STAGES LFSR WITH PHASE SHIFTER.

Circuit	ATPG				Encode		CPU (sec.)
	%FC	%FE	# Patterns	CPU(sec.)	%FC	# Seeds	
b14	96.06	99.87	939	0.38	32.72	203	1.18
b15	86.59	92.18	1,495	2.73	72.27	1,035	4.18
b17	92.00	95.78	7,540	7.59	77.00	6,008	14.55
b18*	97.99	99.33	24,854	19.41	95.48	23,999	7,182.21
b19_6*	96.46	97.99	51,085	77.42	93.51	49,102	14,759.38
b19_13*	96.39	97.96	50,215	80.91	94.19	48,585	13,272.57
b19_22	96.34	97.93	49,709	75.12	94.43	48,142	118.48
b20	95.65	99.89	1,509	0.79	49.60	579	2.40
b21	94.46	98.59	1,689	0.93	21.14	281	2.38
b22	95.73	99.84	1,928	1.05	75.17	1,281	3.39

Table VII
RESULTS OF THE PROPOSED METHOD FOR 100 STAGES LFSR WITH PHASE SHIFTER.

Circuit	%FC	%FE	CPU(sec.)	#Seeds
b14	94.20	99.08	1.16	594
b15	86.60	94.58	8.95	503
b17	90.88	97.47	16.97	2,471
b18	81.28	82.69	367.49	5,912
b19_6	83.88	85.49	913.45	12,853
b19_13	91.56	93.20	382.77	15,735
b19_22	93.75	95.42	261.10	16,695
b20	91.89	97.03	6.88	978
b21	80.63	97.22	15.22	961
b22	93.64	97.60	7.14	1,318

number of seeds dramatically.

Finally, we evaluate seeds generated by our method by fault coverage for RPRFs which is achieved with multiple test pattern expansion per seed. Table VIII shows fault coverage for seeds generated by the conventional method and our method for the LFSR of 100 stages without phase shifter. The number in each column under “Conventional” and “Proposed” show the number of expanded patterns per seed for the respective method. For b18, b19_6, b19_13 and b19_22, fault simulation could not be performed because the execution time and the disk space were limited. For the conventional method, increasing the number of expansion increased fault coverage, i.e., the pseudo-randomly generated test patterns were effective. In contrast, for the proposed method, the pseudo-randomly generated test patterns did not work, i.e., the proposed method was able to generate quality seeds.

Practically, it is difficult to reseed using all the seeds generated because of the limitation of the test application time. We evaluate achievable fault coverage using a limited number of seeds. First, 128 seeds were selected based on three heuristics and were expanded to 128 test patterns per seed. Then, fault coverage for RPRFs was calculated for the test patterns. The result of this experiment is shown in Table IX. Columns “Original order”, “Random ordering” and “FC ordering” show the heuristics the generation order, a randomly selected order and the order of the fastest fault

Table VIII
FAULT COVERAGE FOR SINGLE/MULTIPLE TEST PATTERN EXPANSION
FROM EACH SEED.

Circuit	Conventional					Proposed				
	1	16	32	64	128	1	16	32	64	128
b14	36.16	37.69	39.32	40.73	41.89	96.06	96.06	96.06	96.06	96.06
b15	59.69	60.84	61.47	62.11	63.05	74.75	74.77	74.80	74.83	74.90
b17	47.40	53.22	56.14	59.05	61.18	81.63	81.66	81.69	81.72	81.72
b20	39.35	41.36	42.72	44.84	46.35	90.30	90.34	90.34	90.34	90.34
b21	3.15	3.98	4.49	5.43	7.55	76.33	76.33	76.33	76.33	76.33
b22	58.06	60.22	61.03	61.32	62.01	95.21	95.22	95.23	95.23	95.23

Table IX
FAULT COVERAGE BY RESEEDING (128 SEEDS SELECTED / 128
PATTERN EXPANSION PER SEED).

Circuit	Original order			Random ordering			FC ordering		
	SAT	G-J	Prop	SAT	G-J	Prop	SAT	G-J	Prop
b14	37.17	37.69	62.35	33.23	38.18	60.93	41.80	42.79	67.60
b15	50.84	49.14	60.59	52.15	47.05	57.44	62.54	64.87	68.57
b17	38.59	37.49	48.25	36.89	40.61	51.41	55.11	58.47	66.08
b18	30.75	32.41	30.66	29.77	31.46	31.75	N/A	N/A	N/A
b19_6	27.41	26.85	28.04	26.36	26.96	28.36	N/A	N/A	N/A
b19_13	26.55	27.75	28.67	26.64	27.71	26.93	N/A	N/A	N/A
b19_22	25.05	27.41	28.80	27.71	27.08	26.78	N/A	N/A	N/A
b20	35.96	36.49	49.80	34.53	36.83	51.45	44.64	44.46	65.36
b21	7.55	8.66	29.00	7.55	8.66	23.69	7.55	8.66	57.70
b22	39.72	37.17	51.80	32.40	32.08	39.66	54.75	51.45	69.41

coverage rising, respectively. For the last one, since the 128 expanded test patterns from each seed were simulated, we could not obtain the order for the larger circuits b18, b19_6, b19_13 and b19_22. Columns “SAT”, “G-J” and “Prop” show the conventional method with MiniSAT and with the Gauss-Jordan elimination and the proposed method, respectively. Notice that for b21, 56 seeds were used for the conventional method because only 56 seeds were generated by the method. As shown in the table, the proposed method achieved higher fault coverage for most circuits for Original order and Random ordering. For FC ordering, the proposed method achieved the highest fault coverage among these methods for every circuit.

VI. CONCLUSION

In this paper, we proposed a method of seed generation for LFSR-based deterministic/pseudo-random testing. The method supports phase shifters. The effectiveness of the proposed method has shown through experiments provided in this paper. The proposed method can generate quality seeds and the number of seeds to achieve a fault coverage is smaller than the conventional method. By using the seeds generated by our method, test volume and test application time can be reduced.

Seed generation complexity of the proposed method seems to depend on LFSR and phase shifter structure. However, as shown in the experiments, seeds generated by the proposed method can achieve very high fault coverage for an LFSR with simple structure. Furthermore, the combination

of the conventional and proposed method is useful or the proposed method can generate seeds for the faults missed during the encoding process of the conventional method.

ACKNOWLEDGMENTS

This work is supported by VLSI Design and Education Center(VDEC), the University of Tokyo in collaboration with Synopsys, Inc. and in part by JSPS KAKENHI Grant Number 26330067.

REFERENCES

- [1] M. Abramovici, M. A. Breuer and A. D. Friedman: *Digital Systems Testing and Testable Design*, IEEE Press, 1990.
- [2] N.-C. Lai and S.-J. Wang: “A reseeding technique for LFSR-based BIST application,” in *Proc. of the 11th Asian Test Symposium*, pp. 200–205, 2002.
- [3] B. Konemann: “LFSR-coded test patterns for scan designs,” in *Proc. of European Test Conference*, pp. 237–242, 1991.
- [4] S. Venkataraman, J. Rajski, S. Hellebrand and S. Tarnick: “An efficient BIST scheme based on reseeding of multiple polynomial linear feedback shift registers,” in *Proc. of International Conference on Computer-Aided Design*, pp. 572–577, 1993.
- [5] L.-T. Wang, X. Wen, H. Furukawa, F.-S. Hsu, S.-H. Lin, S.-W. T Tsai, K. S. Abdle-Hafez and S. Wu: “VirtualScan: A new compressed scan technology for test cost reduction,” in *Proc. of International Test Conference*, pp. 916–925, 2004.
- [6] T. Moriyasu and S. Ohtake: “A method of LFSR seed generation for scan-based BIST using constrained ATPG,” in *Proc. of Seventh International Conference on Complex, Intelligent, and Software Intensive Systems (CISIS)*, pp. 755–759, 2013.
- [7] F. Corno, M. S. Reorda and G. Squillero: “RT-level ITC 99 benchmarks and first ATPG results,” *IEEE Design & Test of Computers*, Vol. 17, No. 3, pp. 44–53, 2000.
- [8] P. Alfke: “Efficient shift registers, LFSR counters, and long pseudo-random sequence generators,” *XILINX Application Note*, Vol. XAPP 052, 1996.
- [9] <http://minisat.se/>.
- [10] J. Rajski, N. Tamarapalli and J. Tyszer: “Automated synthesis of phase shifters for built-in self-test applications,” in *Proc. of International Test Conference*, pp. 1047–1056, 1998.
- [11] C. V. Krishna and N. A. Toubia: “Reducing test data volume using lfsr reseeding with seed compression,” in *Proc. of Int. Test Conf.*, pp. 321–330, 2002.